Architecting a Localized Agentic RAG Ecosystem: Integrating GLM-4, LanceDB Namespaces, Iceberg REST, and DLT Hub via Model Context Protocol

The paradigm of enterprise artificial intelligence is currently undergoing a structural and economic realignment. As organizations and individual practitioners encounter the financial friction of subscription-based cloud platforms—such as the premium tiers required for Z.AI coding plans—and the continuous, unpredictable drain of API credits from commercial providers like Gemini or OpenAI, the imperative to transition toward decentralized, local AI deployments has intensified. This transition is not merely a cost-saving measure designed to circumvent recurring subscription fees; it represents a fundamental shift toward absolute data sovereignty, zero-latency inference, and highly customizable agentic workflows. Emulating advanced cloud-native features, specifically autonomous PDF reading, local file indexing, and context-aware coding assistance, requires a meticulously orchestrated stack of open-source technologies operating in absolute synergy.

This comprehensive research report provides an exhaustive, architectural blueprint for constructing a completely localized Retrieval-Augmented Generation (RAG) system deployed on an Apple Silicon ecosystem. Specifically, the target hardware is a MacBook equipped with 48 GB of unified memory. By synthesizing the profound inferential capabilities of locally hosted GLM-4 models in the GPT-Generated Unified Format (GGUF), the automated data extraction and normalization prowess of the Data Load Tool (DLT) Hub, the multimodal storage efficiency of LanceDB, the strict governance of the Apache Iceberg REST Catalog via the novel Lance Namespace architecture, and the seamless interoperability of the Model Context Protocol (MCP), a system can be designed that rivals proprietary cloud offerings. The architecture detailed herein entirely eliminates the reliance on external cloud telemetry, ensuring that no Gemini credits or commercial API tokens are wasted on arranging, indexing, or querying the local file system.

Hardware Constraints and the Apple Silicon Unified Memory Architecture

The foundation of any localized AI architecture is dictated by the physical constraints and specific topologies of the host hardware. Apple's M-series silicon utilizes a unified memory architecture, which provides a massive advantage for high-bandwidth large language model (LLM) inference compared to traditional discrete PCIe GPUs. In a unified architecture, the central processing unit (CPU) and the graphics processing unit (GPU) share the exact same physical memory pool, eliminating the latency-heavy requirement of transferring tensors

across a PCIe bus. However, managing this shared pool requires precise calibration, particularly when deploying complex agentic RAG systems that require concurrent operation of an LLM, a vector database, an orchestration framework, and an integrated development environment (IDE).

A system equipped with 48 GB of unified memory cannot allocate the entirety of its capacity to the machine learning model. The macOS operating system, background daemons, the DLT orchestration stack, the Iceberg REST container (such as Apache Polaris or Gravitino), and the MCP server infrastructure require a baseline allocation. Empirical testing on Apple Silicon indicates that the operating system and essential applications effortlessly consume between 8 GB and 12 GB of memory under active development loads.[1] Consequently, the safe, non-paging upper boundary for model weights and the Key-Value (KV) cache combined is approximately 36 GB.[1] Exceeding this boundary forces the macOS kernel to page memory to the NVMe solid-state drive, resulting in a catastrophic degradation of inference speed, often dropping from tens of tokens per second to fractions of a token per second.

When dealing with deep reasoning models capable of matching the capabilities of a Z.AI coding assistant, such as the 32-billion parameter GLM-4-32B or the highly optimized GLM-4.7-Flash, understanding these memory boundaries is the primary determinant of system stability. The memory allocation must be mathematically distributed between the static model weights (determined by the quantization format) and the dynamic KV cache (determined by the context window length required for reading extensive PDF documents).

Core Reasoning Engine: Evaluating and Deploying GLM-4 Models

The core reasoning engine of this localized architecture relies on the GLM-4 family of large language models. Developed to handle complex multi-step reasoning, multilingual processing, and deep code generation, the GLM-4 architecture is particularly well-suited for autonomous agentic tasks.[3] Deploying these massive models locally requires advanced quantization—reducing the precision of the model's weights from 16-bit floating-point (BF16) to lower bit-width representations. This drastically shrinks the memory footprint at the cost of marginal perplexity degradation.[4]

Quantization Topologies and GGUF Selection Criteria

The GLM-4-32B model, at full BF16 precision, requires 65.1 GB of memory purely for the weights, which immediately exceeds the hardware limits of the target 48 GB machine.[3] Therefore, advanced quantization methods, specifically those provided by the llama.cpp imatrix (importance matrix) optimizations, must be utilized.[6] The importance matrix technique evaluates the model against a calibration dataset to determine which specific weights are most critical for preserving reasoning logic, allowing those specific weights to be quantized less aggressively than less important pathways.

The following table illustrates the exact memory footprint for various GGUF quantizations of the GLM-4-32B-0414 model, evaluating their mathematical viability on a 48 GB macOS system

assuming a strict 36 GB ceiling for AI operations:

Quantization Format	Average Bits per Weight	File Size (GB)	Quality Retention Profile	48GB Mac Viability (Including Required KV Cache)
BF16	16.0	65.14	Baseline Truth (100%)	Mathematically Impossible (OOM)
Q8_0 / Q8_K_XL	8.5	34.62 - 39.50	Extremely High Quality	Unviable (Insufficient memory for context window)
Q6_K	6.6	26.73 - 28.70	Near Perfect	Viable for highly restricted, short-context prompts
Q5_K_M / Q5_K_XL	5.5	23.10	High Quality	Viable for medium-context document querying
Q4_K_M	4.8	19.70	Good Quality (Recommended)	Optimal for maximal context window and deep RAG
Q4_0	4.5	18.60	Legacy Format	Viable, but inferior token logic compared to K-quants
IQ4_XS	4.3	17.60	Decent Quality	Aggressive saving, slight degradation in coding logic
Q3_K_M	3.5	15.89	Noticeable Logic Loss	Sub-optimal for complex codebase refactoring

For the specified 48 GB configuration, the Q4_K_M (19.70 GB) variant represents the optimal intersection of reasoning fidelity and memory economy.[3] The utilization of 19.70 GB of unified memory for the model weights leaves a critical surplus of approximately 16.3 GB within the safe operational threshold of the 48 GB Apple Silicon architecture. This surplus is not merely an idle buffer; it is the fundamental physical requirement for instantiating the massive Key-Value (KV) cache necessary for the ingestion of substantial PDF documents without triggering memory

paging.
Alternatively, smaller variants such as the GLM-4-9B or GLM-4.1V-9B models can be utilized. The 9-billion parameter models require drastically less memory, utilizing only 6.25 GB in the Q4_K_M format and 8.26 GB in the Q6_K format.[7] While these smaller models execute much faster on Apple Silicon, and leave vast amounts of RAM available for other processes, their inherent reasoning capabilities for complex codebase refactoring, intricate logical deductions, or dense PDF analysis generally fall short of the 32B tier. When the objective is to entirely replace the premium reasoning capabilities of Z.AI, the 32B Q4_K_M model is the preferred deployment target.[3] Furthermore, models quantized by community developers such as unsloth, zai-org, and bartowski often include refined imatrix datasets that dramatically improve the stability of the model at the 4-bit level.[6]

Inference Engines and the KV Cache Mathematics

The execution of GGUF models on Apple Silicon relies heavily on the underlying inference engine. The two dominant paradigms for executing local weights are llama.cpp (and its daemonized derivatives like Ollama or LM Studio) and MLX (Apple's proprietary machine learning framework optimized for unified memory).[2]
While MLX optimizations can yield extraordinary inference speeds—sometimes pushing 60 to 80 tokens per second for mid-sized models on M-series chips—it requires specific model formats (such as .npz or MLX-safetensors) and handles KV caching quite aggressively.[12] Recent community testing indicates that certain MLX backends consume up to four times the expected memory for the KV cache due to current implementation inefficiencies, leading to rapid Out-Of-Memory (OOM) errors during long-context tasks.[13] Furthermore, some MLX backend implementations natively cap the default context window at 2,048 tokens because the indexer fails to read the long-context hints from the GLM configuration files. This artificially restricts the model's ability to read PDFs unless explicitly overridden in the config.json via the max_position_embeddings parameter (setting it to 32,768) or through strict inference flags at load time.[14]
Given these nuances, for a highly stable, always-on GGUF deployment that integrates seamlessly with background RAG processes and an MCP server, utilizing Ollama or a raw llama.cpp server as the backend daemon is highly recommended.[11] Ollama provides a robust REST API, automatically handles model loading and unloading from unified memory, and integrates natively with local embedding providers.[11]
When emulating an exhaustive PDF reader, the context window—the amount of text the model can retain in its active memory buffer during a single prompt—becomes the primary system bottleneck. The KV cache stores the key and value tensors of previous tokens to prevent redundant recalculations during autoregressive generation. The memory requirement for the KV cache can be modeled mathematically. For a transformer model with L layers, hidden dimension D, and N_h attention heads, the memory M_{KV} required per token in bytes (assuming standard 16-bit precision for the cache) is roughly:

$$M_{KV} = 4 \times L \times D$$
Scaling the context window of the GLM-4-32B model to 32,768 tokens (32k) can consume upwards of 18 GB of memory purely for the KV cache, depending on the implementation of Flash Attention.[14] Given the 48 GB system limit, utilizing a Q4_K_M model (19.7 GB) plus a full 32k KV cache (18 GB) results in exactly 37.7 GB of utilization. This fits tightly within the ~36 GB to 40 GB safe boundary of the MacBook, proving that deep document analysis without cloud reliance is mathematically and physically viable on this specific hardware topology.[1]

Automated Data Engineering: Extraction and Normalization via DLT Hub

To emulate a robust file indexer and PDF reader, raw unstructured data must be continuously monitored, extracted, normalized, transformed into vector embeddings, and loaded into a vector database. Manually arranging the file system or relying on bespoke Python scripts leads to memory leaks, state management failures, and brittle pipelines. The Data Load Tool (DLT) is an open-source Python library designed to automate the most tedious aspects of this pipeline: schema evolution, memory management during extraction, and scalable micro-batching.[16]
Using DLT prevents the memory crashes typically associated with parsing massive local file systems. DLT controls the maximum RAM utilized during extraction by processing data streams through generators and iterators rather than loading entire multi-gigabyte datasets into memory simultaneously.[16] This ensures that the DLT extraction process does not compete for the unified memory critically needed by the GLM-4 inference engine.

The Filesystem Source and Unstructured Data Handling

The first stage of the localized indexer is the source pipeline. DLT provides verified, highly optimized sources for both local filesystems and unstructured data APIs.[18] The DLT filesystem source natively accesses local directories, acting as an intelligent iterator that yields FileItem objects.[20] These objects contain rich metadata—such as file_url, modification_date, mime_type, and size_in_bytes—without eagerly loading the actual file contents into memory.[20]
This metadata-first approach allows the system to monitor vast codebase directories for new or modified PDFs and only process the delta (incremental extraction), saving immense compute cycles.[18]
For the actual parsing of PDFs, DLT integrates with unstructured data processing paradigms. Emulating a sophisticated Z.AI PDF reader requires more than simple text extraction; it requires deep layout awareness to distinguish between paragraphs, multi-column tables, merged cells, and headers. While libraries like pdfminer or pdfplumber offer basic extraction capabilities, they frequently fail on complex, multi-column academic papers or highly formatted API documentation.[21] The optimal approach is to integrate DLT with advanced parsing logic, such as the unstructured library (running entirely locally, bypassing their paid API), which utilizes computer vision heuristics to partition documents into distinct structural elements (e.g.,

separating a title from a data table).[19]
The operational execution involves a sequential protocol where the filesystem resource detects a new PDF.[20] The pipeline filters the file based on its modification date to ensure only new documents are processed.[18] A custom reader function then opens the FileItem object, parses the document using local partitioning models, extracts the text alongside structural metadata, and yields discrete dictionaries representing text chunks.[20]

Vectorization and the LanceDB DLT Adapter

The critical requirement of this specific architecture is the absolute avoidance of external API credits. Therefore, the embedding process—translating the extracted PDF text chunks into dense numerical vectors for semantic search—must also occur locally.[23] Sending text to the Gemini API or OpenAI API defeats the purpose of the localized ecosystem.
DLT provides native, out-of-the-box integration with LanceDB via the lancedb_adapter.[15] LanceDB acts as a highly optimized, columnar vector database.[15] When the lancedb_adapter wraps a DLT resource, it fundamentally alters the destination pipeline, instructing the loader to automatically vectorize designated text fields upon insertion.[15]
To configure this for local execution, the DLT secrets and configuration file (typically located at ~/.dlt/secrets.toml) must be explicitly modified to point to a local embedding provider rather than a commercial API. Since Ollama is already hosting the GLM-4 reasoning model, it can simultaneously host a lightweight, highly specialized embedding model, such as nomic-embed-text or mxbai-embed-large.[15] These embedding models require minimal memory (often under 1 GB) and produce high-quality vectors extremely rapidly on Apple Silicon.
The exact configuration required in the secrets.toml file enforces this strict localization:

Ini, TOML

```
[destination.lancedb]
lance_uri = ".lancedb"
embedding_model_provider = "ollama"
embedding_model = "nomic-embed-text"
embedding_model_provider_host = "http://localhost:11434"

[destination.lancedb.credentials]
# Authentication parameters are explicitly left blank to bypass API key requirements for local Ollama endpoints
```

Within the main Python execution script, the adapter is applied seamlessly around the resource yielding the parsed PDF chunks:

Python

```
import dlt
from dlt.destinations.adapters import lancedb_adapter

pipeline = dlt.pipeline(
   pipeline_name="local_pdf_indexer",
   destination="lancedb",
   dataset_name="codebase_knowledge"
)

# The 'embed' argument instructs LanceDB to vectorize the 'text_content' field
info = pipeline.run(
   lancedb_adapter(
      pdf_chunks_resource(),
      embed="text_content"
   ),
   table_name="documents"
)
```

By passing embed="text_content", DLT automatically routes the parsed PDF text to the local Ollama embedding endpoint via local network calls, generates the high-dimensional vectors, and loads the raw text, the structural metadata, and the vector arrays directly into the LanceDB destination.[15] This pipeline perfectly emulates a cloud-based file indexer without a single byte of data leaving the MacBook, operating entirely free of recurring API costs.

Architecting the Vector Lakehouse: LanceDB and the Namespace Abstraction

While storing vectors in a local LanceDB instance solves the immediate retrieval problem for a basic RAG application, enterprise-grade AI applications require strict data governance, version control, and seamless interoperability with other analytical and compute tools. Storing isolated .lancedb directories without a centralized catalog quickly leads to fragmented file systems and metadata rot. This is where the LanceDB "Namespace" architecture, and its explicit compatibility with the Apache Iceberg REST Catalog, becomes a transformative architectural component.

The Divergence of BI and AI Workloads

Historically, data lakehouses have been dominated by the Apache Iceberg table format, which provides crucial capabilities such as ACID transactions, dynamic schema evolution, and time-travel querying for structured, analytical Business Intelligence (BI) workloads.[25] However,

Iceberg was fundamentally designed for structured, tabular data. It inherently struggles with the multimodal, high-dimensional arrays (vectors) required by modern Artificial Intelligence (AI) and Machine Learning (ML) workloads.[25]

LanceDB, built upon the open-source Lance columnar format, was engineered from the ground up to address these exact AI bottlenecks. Lance provides native support for high-dimensional vector search, maintains extremely fast random access capabilities (unlike traditional columnar formats like Parquet, which require scanning large blocks), and enables zero-copy reads via Apache Arrow.[26]

The prevailing industry challenge has been integrating the high-performance AI storage format of Lance with the established, trusted metadata governance frameworks of Iceberg. To resolve this tension, the Lance open-source community introduced the "Lance Namespace" architecture.[28]

The Lance Namespace Conceptual Model

Instead of attempting to force Lance vector data into Iceberg's rigid metadata trees—which require multiple, latency-heavy hops including a catalog lookup, metadata file retrieval, manifest list parsing, and manifest file loading before finally accessing the data—Lance utilizes a decoupled "Namespace" design.[25]

The Lance Namespace is a thin, standardized abstraction layer—defined formally as a client specification—that allows Lance tables to be stored, tracked, and managed by any underlying catalog service.[28] It is intentionally termed a "Namespace" rather than a "Catalog" because it acts as a universal interface, adapting to whatever schema, metastore, or metalake the user prefers.[29] This architecture allows developers to organize their massive datasets hierarchically, utilizing a logical Database $\rightarrow$ Namespace $\rightarrow$ Table structure.[30]

When LanceDB operates within an Iceberg ecosystem, it utilizes integrations maintained in the dedicated lance-namespace-impls repository.[31] This repository contains implementations that allow the core Lance client (in Python, Java, or Rust) to communicate directly with external catalog systems, translating Lance namespace operations into the specific API calls expected by the destination catalog.[29]

Integration via the Apache Iceberg REST Catalog

The Apache Iceberg REST Catalog specification is a vendor-neutral OpenAPI standard defining exactly how external compute services communicate with a catalog over standard HTTP/HTTPS protocols.[33] To emulate a highly robust, governed local environment on the MacBook, a lightweight open-source REST catalog must be deployed. The premier options for this are Apache Polaris (incubating) or Apache Gravitino.[33]

Deploying these catalogs locally is achieved via Docker.[34] Apache Polaris, originally contributed by Snowflake, offers enterprise-grade data catalog capabilities via simple REST APIs.[36] However, it is a Java-based application that requires significant resource allocation; deploying Polaris requires a minimum of 4 GB of RAM allocated to the Docker container, as deploying it with typical default limits (e.g., 2 GB) causes the Java Virtual Machine to choke and fail during

startup.[37] Alternatively, Apache Gravitino provides an exceptionally robust, next-generation REST catalog service that also seamlessly supports the Iceberg REST interface, acting as a unified metadata lake.[35]

The integration mechanism between the LanceDB Python client and the Iceberg REST Catalog is an elegant exercise in metadata proxying and object mapping. When a Lance table is registered via the Iceberg REST API, the Lance Namespace implementation creates a "companion" Iceberg table at the exact same physical storage location.[39]

This companion table utilizes a fascinating architectural workaround known as a "dummy schema." The schema explicitly contains only a single, nullable string column named dummy.[39] Because the table format conforms to Iceberg's baseline metadata requirements, the Iceberg REST catalog successfully tracks the asset without attempting to parse the complex, incompatible multimodal Lance vectors. To differentiate this table from standard BI tables, the Lance integration injects specific metadata into the Iceberg table's properties map—most notably, the key-value pair table_type=lance (case-insensitive).[39]

The following table details the specific REST API operations utilized by the Lance Namespace implementation to manage assets within the Iceberg REST Catalog:

Namespace Operation	**HTTP Method**	**API Endpoint Pattern**	**Payload / Mechanism Description**
CreateNamespace	POST	/v1/{prefix}/namespaces	Constructs a CreateNamespaceRequest containing the namespace array and properties.
DeclareTable	POST	/v1/{prefix}/namespaces/{namespace}/tables	Constructs a CreateTableRequest containing the dummy schema, storage location, and table_type=lance property.
DescribeTable	GET	/v1/{prefix}/namespaces/{namespace}/tables/{table}	Verifies table_type=lance and returns the physical storage location, bypassing standard schema loads.
ListTables	GET	/v1/{prefix}/namespaces/{namespace}/tables	Retrieves all tables in a namespace, filtering the response to only return tables with the lance property.
DeregisterTable	DELETE	/v1/{prefix}/namespace	Sends request with

		s/{namespace}/tables/{table}	purgeRequested=false to remove metadata without destroying the underlying Lance vector data.

To initialize this integration within the DLT pipeline or a standalone Python script, the developer must configure the Lance client to point to the local Docker container hosting the REST catalog. This requires setting specific properties: the endpoint (the URL of the Iceberg REST server, e.g., http://localhost:8181), the root (the default storage location on the MacBook where the .lance files reside), and optional auth_token or credential strings if the local Polaris/Gravitino instance has security enabled.[39]

When a query is executed, the compute engine queries the Iceberg REST catalog, which instantly returns the physical directory location of the Lance data.[39] The local compute engine then bypasses the complex Iceberg manifest trees entirely and reads the zero-copy Arrow arrays directly from the local disk.[39] By implementing this architecture, the user achieves the strict organizational hierarchy and catalog standard of Apache Iceberg, combined with the lightning-fast, high-dimensional vector retrieval capabilities of LanceDB, all running locally without relying on external cloud infrastructure.[25]

Orchestrating Agentic RAG: The Model Context Protocol (MCP)

Having established a robust reasoning engine (the locally hosted GLM-4-32B model) and a highly structured, automated knowledge base (DLT parsing PDFs into LanceDB, governed by the Iceberg REST Catalog), the final architectural requirement is bridging the gap between the LLM and the data. The goal is to entirely replace the Z.AI coding plan, which utilizes proprietary remote servers to allow its cloud models to read web pages and index project files.[40] This autonomous capability is achieved through the Model Context Protocol (MCP).

MCP is an open standard, championed by organizations like Anthropic, designed to unify how AI applications access external context.[41] Instead of hardcoding custom integrations, API wrappers, and bespoke Python scripts for every new tool or database, MCP acts as a universal, standardized interface—frequently described by engineers as a "USB-C port for AI applications".[42] An MCP server exposes specific capabilities (categorized as Tools, Resources, and Prompts) over a standardized JSON-RPC protocol, which any MCP-compatible client (such as the Claude Desktop application, the Zed Editor, or a custom local IDE) can seamlessly consume.[41]

Replacing the Z.AI PDF Reader and File Indexer

To emulate Z.AI's features locally without subscription costs, open-source MCP server implementations must be deployed that interface directly with the LanceDB vector store. The

open-source community has rapidly developed several highly capable MCP servers designed explicitly for local RAG workflows. Notable implementations include lance-mcp by adiom-data, document-mcp by yairwein, and smart-search by ekmungi.[45]

The document-mcp project provides a comprehensive, production-ready blueprint for emulating a local file indexer. Written entirely in Python, it inherently utilizes the watchdog library to continuously and passively monitor configured local folders for new, deleted, or modified documents (including PDFs, Markdown, and Word files).[46] It operates in perfect harmony with the DLT pipeline described previously. When the DLT pipeline drops newly processed, chunked text into LanceDB, the MCP server acts as the intelligent intermediary. It maintains a persistent connection to the local LanceDB vector store and exposes search tools directly to the LLM.[46]

An MCP server registers specific "Tools" that the LLM is informed of and can decide to utilize autonomously based on the user's prompt. For example, the smart-search MCP server exposes a highly refined tool titled knowledge_search.[47] This tool accepts parameters defined by a strict, pre-defined JSON schema:

- **query** (string, required): The natural language search term generated by the LLM based on its deduction of what information is missing from its context.
- **limit** (integer, optional): The maximum number of vector chunks to return from the database to prevent overwhelming the context window.
- **doc_types** (list of strings, optional): Filters to restrict the search to specific file extensions, such as ["pdf"] or ["md"].[47]

The Agentic Execution Flow

The power of this architecture lies in its autonomy, often referred to as Agentic RAG. When the user queries the locally hosted GLM-4 model via an interface like the Zed Editor with a complex prompt—such as, "Analyze the architectural diagrams in the recently downloaded PDF regarding our new microservices, and write a Python script that implements the API flow described on page 12"—the model immediately recognizes that its pre-trained weights do not contain this proprietary information.

Because the Zed Editor (acting as the MCP client) has previously queried the MCP server and provided the GLM-4 model with the schema for the knowledge_search tool, the LLM halts its standard text generation. Instead, it formulates a highly specific search query and triggers a tool call.[42]

The Zed Editor passes this JSON-RPC tool call to the local MCP server over standard input/output (stdio) streams.[49] The MCP server receives the query, silently pings the local Ollama instance to embed the query text into a mathematical vector, and executes a lightning-fast similarity search against the governed LanceDB tables.[47] LanceDB returns the most semantically relevant text chunks, along with critical structural metadata such as the source file name, page numbers, and section heading paths.[47]

The MCP server formats these results into a unified, structured JSON block and returns them to the LLM through the client.[47] The GLM-4 model then ingests this highly targeted retrieved

context into its massive KV cache, synthesizes the information, and generates a coherent, highly accurate Python script, effectively replacing the need for an external, paid service.[42]

Client Configuration and Final Orchestration

To wire the MCP server to the client application, a simple configuration file is required. Whether utilizing the Claude Desktop application or an MCP-compatible IDE like Zed, the configuration file (e.g., claude_desktop_config.json or Zed's settings.json) is modified to launch the MCP server as a background subprocess whenever the editor is opened:

JSON

```
{
  "mcpServers": {
    "local_lancedb_indexer": {
      "command": "uv",
      "args": [
        "run",
        "python",
        "-m",
        "document_mcp",
        "--db-path",
        "~/.lancedb"
      ]
    }
  }
}
```

This minimal configuration instructs the client environment to seamlessly invoke the Python process running the MCP server, passing the physical path to the local LanceDB instance governed by the Iceberg REST catalog.[51] Because all components—the GLM-4 GGUF inference engine, the DLT extraction and normalization pipeline, the LanceDB vector store, the Polaris Iceberg REST catalog, and the MCP server—are executing entirely locally on the MacBook's unified memory, the entire interaction lifecycle occurs with near-zero latency. Most importantly, it guarantees absolute privacy, zero cloud telemetry, and absolutely no API credit consumption, successfully architecting a federated, localized ecosystem that exceeds the capabilities of commercial, subscription-based coding plans.[45]

Works cited

1. Trying to run llama-3.3 70B 34.59GB on my M4 MBP with 48GB ram has strange peaks then a wait. Fairly slow inference run in LM Studio. What is going on? - Reddit, accessed on March 20, 2026,

https://www.reddit.com/r/LocalLLM/comments/1kfuztg/trying_to_run_llama33_70b_3459gb_on_my_m4_mbp/
2. Run GLM-4.7-Flash Locally: Ollama, Mac & Windows Setup | by WaveSpeedAI - Medium, accessed on March 20, 2026, https://medium.com/@social_18794/run-glm-4-7-flash-locally-ollama-mac-windows-setup-2ab77bdb26b5
3. unsloth/GLM-4-32B-0414-GGUF - Hugging Face, accessed on March 20, 2026, https://huggingface.co/unsloth/GLM-4-32B-0414-GGUF
4. Self-Hosting Your First LLM | Towards Data Science, accessed on March 20, 2026, https://towardsdatascience.com/self-hosting-your-first-llm/
5. bartowski/THUDM_GLM-4-32B-0414-GGUF - Hugging Face, accessed on March 20, 2026, https://huggingface.co/bartowski/THUDM_GLM-4-32B-0414-GGUF
6. bartowski/zai-org_GLM-4.6-GGUF - Hugging Face, accessed on March 20, 2026, https://huggingface.co/bartowski/zai-org_GLM-4.6-GGUF
7. second-state/glm-4-9b-chat-GGUF - Hugging Face, accessed on March 20, 2026, https://huggingface.co/second-state/glm-4-9b-chat-GGUF
8. unsloth/GLM-4.1V-9B-Thinking-GGUF - Hugging Face, accessed on March 20, 2026, https://huggingface.co/unsloth/GLM-4.1V-9B-Thinking-GGUF
9. mradermacher/glm-4-9b-chat-GGUF - Hugging Face, accessed on March 20, 2026, https://huggingface.co/mradermacher/glm-4-9b-chat-GGUF
10. zai-org/GLM-4.7-Flash · Possible to run this in 8GB VRAM + 48GB RAM? - Hugging Face, accessed on March 20, 2026, https://huggingface.co/zai-org/GLM-4.7-Flash/discussions/46
11. llama.cpp vs. ollama: Running LLMs Locally for Enterprises - Picovoice, accessed on March 20, 2026, https://picovoice.ai/blog/local-llms-llamacpp-ollama/
12. GLM-4.7-Flash: The Ultimate 2026 Guide to Local AI Coding Assistant - Medium, accessed on March 20, 2026, https://medium.com/@zh.milo/glm-4-7-flash-the-ultimate-2026-guide-to-local-ai-coding-assistant-93a43c3f8db3
13. Glm 4.7 flash, insane memory usage on MLX (LM studio) : r/LocalLLaMA - Reddit, accessed on March 20, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1qiqo54/glm_47_flash_insane_memory_usage_on_mlx_lm_studio/
14. mlx-community/GLM-4-32B-0414-4bit · 2048 context length? - Hugging Face, accessed on March 20, 2026, https://huggingface.co/mlx-community/GLM-4-32B-0414-4bit/discussions/1
15. LanceDB | dlt Docs - dltHub, accessed on March 20, 2026, https://dlthub.com/docs/dlt-ecosystem/destinations/lancedb
16. Pipeline tutorial | dlt Docs - dltHub, accessed on March 20, 2026, https://dlthub.com/docs/build-a-pipeline-tutorial
17. Extracting data with dlt - DEV Community, accessed on March 20, 2026, https://dev.to/cmcrawford2/extracting-data-with-dlt-9hl
18. Filesystem source | dlt Docs - dltHub, accessed on March 20, 2026, https://dlthub.com/docs/dlt-ecosystem/verified-sources/filesystem/basic
19. Unstructured Python API Docs | dltHub, accessed on March 20, 2026,

https://dlthub.com/context/source/unstructured
20. Advanced filesystem usage | dlt Docs - dltHub, accessed on March 20, 2026, https://dlthub.com/docs/dlt-ecosystem/verified-sources/filesystem/advanced
21. Unstructured PDF Text Extraction - Khadija Mahanga - Medium, accessed on March 20, 2026, https://medium.com/khadija-mahanga/unstructured-pdf-extraction-series-a8ba04da767f
22. Process PDFs in Python: Step-by-Step Guide | Unstructured, accessed on March 20, 2026, https://unstructured.io/blog/how-to-process-pdf-in-python
23. LanceDB | dlt Docs - dltHub, accessed on March 20, 2026, https://dlthub.com/docs/dlt-ecosystem/destinations/lancedb#lancedb-adapter-with-local-embeddings
24. Ollama - LanceDB, accessed on March 20, 2026, https://docs.lancedb.com/integrations/embedding/ollama
25. The Future of Open Source Table Formats: Apache Iceberg and Lance - LanceDB, accessed on March 20, 2026, https://lancedb.com/blog/the-future-of-open-source-table-formats-iceberg-and-lance/
26. Lance REST service - Apache Gravitino, accessed on March 20, 2026, https://gravitino.apache.org/docs/1.1.0/lance-rest-service/
27. Lance - Daft Documentation, accessed on March 20, 2026, https://docs.daft.ai/en/stable/connectors/lance/
28. From BI to AI: A Modern Lakehouse Stack with Lance and Iceberg - LanceDB, accessed on March 20, 2026, https://lancedb.com/blog/from-bi-to-ai-lance-and-iceberg/
29. Lance Namespace is an open specification for describing access and operations against a collection of tables in a multimodal lakehouse - GitHub, accessed on March 20, 2026, https://github.com/lance-format/lance-namespace
30. Proposal: Introduce Catalog for LanceDB · Issue #3257 · lance-format/lance - GitHub, accessed on March 20, 2026, https://github.com/lancedb/lance/issues/3257
31. Lance SDK v1.0.0, 🗓️ 1st Lance Community Sync, Wikisearch - LanceDB, accessed on March 20, 2026, https://lancedb.com/blog/newsletter-december-2025/
32. Lance Namespace Impls - Lance, accessed on March 20, 2026, https://lance.org/community/project-specific/namespace-impls/
33. Iceberg REST catalog service - Apache Gravitino, accessed on March 20, 2026, https://gravitino.apache.org/docs/0.6.1-incubating/iceberg-rest-service
34. Quick start with Apache Iceberg and Polaris - Dremio, accessed on March 20, 2026, https://www.dremio.com/blog/quick-start-with-apache-iceberg-and-apache-polaris-on-your-laptop-quick-setup-notebook-environment/
35. Gravitino: Next-Gen REST Catalog for Iceberg, and Why You Need It - DEV Community, accessed on March 20, 2026, https://dev.to/datastrato/gravitino-next-gen-rest-catalog-for-iceberg-and-why-y

ou-need-it-53ab
36. A Local Playground for Apache Polaris | by Kamesh Sampath | Snowflake Builders Blog, accessed on March 20, 2026, https://medium.com/snowflake/lets-build-together-a-local-playground-for-apache-polaris-fdcad4485f43
37. How I (Barely) Survived Setting Up Polaris as an Iceberg REST Catalog, accessed on March 20, 2026, https://www.confessionsofadataguy.com/how-i-barely-survived-setting-up-polaris-as-an-iceberg-rest-catalog/
38. Introduction to REST Catalogs for Apache Iceberg | by Datastrato - Medium, accessed on March 20, 2026, https://medium.com/datastrato/introduction-to-rest-catalogs-for-apache-iceberg-5ee4b6d05eaa
39. Apache Iceberg REST Catalog - Lance, accessed on March 20, 2026, https://lance.org/format/namespace/integrations/iceberg/
40. Web Reader MCP Server - Overview - Z.AI DEVELOPER DOCUMENT, accessed on March 20, 2026, https://docs.z.ai/devpack/mcp/reader-mcp-server
41. punkpeye/awesome-mcp-servers - GitHub, accessed on March 20, 2026, https://github.com/punkpeye/awesome-mcp-servers
42. True Agentic RAG: How I Taught Claude to Talk to My PDFs using MCP (Model Context Protocol) | by Alexander Komyagin | Medium, accessed on March 20, 2026, https://medium.com/@adkomyagin/true-agentic-rag-how-i-taught-claude-to-talk-to-my-pdfs-using-model-context-protocol-mcp-9b8671b00de1
43. Unlocking Agentic RAG: A Deep Dive into the LanceDB MCP Server by Alex Komyagin, accessed on March 20, 2026, https://skywork.ai/skypage/en/agentic-rag-lancedb-server/1980834220848885760
44. Model Context Protocol (MCP) in Zed, accessed on March 20, 2026, https://zed.dev/docs/assistant/model-context-protocol
45. LanceDB | Awesome MCP Servers, accessed on March 20, 2026, https://mcpservers.org/servers/adiom-data/lance-mcp
46. yairwein/document-mcp: MCP server to access local documents - GitHub, accessed on March 20, 2026, https://github.com/yairwein/document-mcp
47. smart-search | MCP Servers - LobeHub, accessed on March 20, 2026, https://lobehub.com/mcp/ekmungi-smart-search
48. freespirit/pdfsearch-zed: An MCP server extension for Zed that retrieves relevant pieces from a PDF file - GitHub, accessed on March 20, 2026, https://github.com/freespirit/pdfsearch-zed
49. MCP server | dlt Docs - dltHub, accessed on March 20, 2026, https://dlthub.com/docs/hub/features/mcp-server
50. vurtnec/mcp-LanceDB-node - GitHub, accessed on March 20, 2026, https://github.com/vurtnec/mcp-LanceDB-node
51. LanceDB MCP Server - LobeHub, accessed on March 20, 2026, https://lobehub.com/mcp/ryanlisse-lancedb_mcp

52. Your Personal AI Knowledge Base: A Deep Dive into the Document Indexer MCP Server, accessed on March 20, 2026, https://skywork.ai/skypage/en/personal-ai-knowledge-base/1981251292132986880